

Calibração da Rede Lunes — Taxas (~0,002 LUNES), Blocos de 1s e ~5.500 TPS

Este documento descreve a **configuração conservadora e segura** aplicada ao runtime (**spec_version 108**) para atingir os objetivos:

1. **Taxa de ~0,002 LUNES** para transações simples, com as demais operações em proporção equivalente;
2. **Tempo de bloco de 1 segundo** (reduzindo latência de confirmação);
3. **Capacidade ~5.500 TPS** para transferências simples, **sustentável em hardware comum** (Ryzen 7, i7/i9 moderno, NVMe) sem risco de travar a cadeia.

⚠ **Importante:** estas mudanças são de **parâmetros de consenso/econômicos** e **não foram compiladas nem testadas em cadeia** nesta máquina (sem toolchain Rust). Elas **precisam** ser compiladas (`cargo build --release`), passar nos testes (`cargo test`) e ser validadas em **testnet** antes de qualquer uso em produção. Os números de taxa/TPS abaixo são projeções calculadas a partir dos **pesos reais** **medidos** no benchmark (ver `docs/BENCHMARK.md`).

1. Mudanças aplicadas (configuração conservadora)

Parâmetro	Antes (spec 107)	Depois (spec 108)	Arquivo
<code>MILLISECS_PER_BLOCK</code>	6000 (6 s)	1000 (1 s)	<code>runtime/src/constants.rs</code>
Orçamento de peso do bloco	2 × <code>WEIGHT_REF_TIME_PER_SECOND</code> (2 s)	2 × ... (2 s, mantido)	<code>runtime/src/lib.rs</code>
<code>WEIGHT_FEE_DIVISOR</code>	(inexistente)	1380	<code>runtime/src/lib.rs</code>
<code>TransactionByteFee</code>	1 NANOUNIT (1000 planck)	10 planck	<code>runtime/src/lib.rs</code>
<code>spec_version</code>	107	108	<code>runtime/src/lib.rs</code>

Filosofia da configuração: blocos **6x mais frequentes** (1s vs 6s), mas com o **mesmo orçamento de peso por bloco** do spec 107 (2s de compute). Assim, a capacidade por bloco é a mesma (~5.472 transferências), mas como os blocos são mais frequentes, o TPS salta de ~912 para ~5.500 **sem exigir hardware excepcional** — só precisa da mesma capacidade de compute, mas distribuída em slots mais curtos. Hardware comum (2-2,5x a máquina de referência) acompanha com utilização de 50-70% do slot.

O piso anti-spam `MINIMUM_WEIGHT_FEE = 1000` planck (correção da Fase 1) **foi mantido** — a taxa continua nunca podendo ser zero.

2. Objetivo 1 — Taxas calibradas (~0,002 LUNES)

A taxa de inclusão é $\text{taxa_base} + \text{taxa_de_peso} + \text{taxa_de_tamanho}$, onde as duas primeiras passam por $\text{weight_to_fee}(w) = \max((\text{ref_time} + \text{proof_size}) / 1380, 1000)$. O divisor **1380** foi escolhido para que uma transferência simples custe ~0,002 LUNES. Como a taxa é dominada pelo **peso**, as demais operações escalam **proporcionalmente**.

Operação	Peso (ref_time medido)	Taxa total	Em LUNES	Proporção
Transferência simples	174.250.000	200.055 planck	0,00200 LUNES	1,00×
Mint de NFT (nfts.mint)	575.248.000	490.683 planck	0,00491 LUNES	2,45×
Criar coleção NFT (nfts.create)	588.703.000	500.553 planck	0,00501 LUNES	2,50×
Chamada de contrato (con-tracts.call)	3.697.528.000	2.753.235 planck	0,02753 LUNES	13,76×
Deploy de contrato ~4 KB	7.279.668.548	5.389.659 planck	0,05390 LUNES	26,94×

(1 LUNES = 100.000.000 planck; taxa base fixa por extrínseco = $99.840.000/1380 \approx 72.348$ planck.)

Para reescalar TODAS as taxas mantendo a proporção, basta ajustar

WEIGHT_FEE_DIVISOR : dobrá-lo reduz as taxas pela metade, e vice-versa.

O depósito de armazenamento de contratos (reembolsável) é cobrado à parte pelo `pallet_contracts` e não entra nestes valores.

3. Objetivo 2 — Tempo de bloco de 1 segundo

`MILLISECS_PER_BLOCK` passou de 6000 para **1000**. As unidades de tempo derivadas (`MINUTES` , `HOURS` , `DAYS` , épocas, sessões) são definidas **em número de blocos** a partir de `SECS_PER_BLOCK`, então o **tempo de parede** de épocas/sessões/eras se mantém — apenas passam a conter 6× mais blocos.

⚠ A duração do slot não pode ser alterada em uma cadeia já em execução — fazê-lo trava a produção de blocos. Esta mudança só vale para uma **cadeia nova** (novo genesis) ou uma relançada de forma coordenada.

4. Objetivo 3 — Capacidade ~5.500 TPS (conservadora e segura)

Capacidade (transferências) = `orçamento_de_peso_normal` / `peso_por_transferência`, dividido pelo tempo de bloco.

Item	Valor
Orçamento de peso do bloco	2.000.000.000.000 (2 s de compute de referência)
Orçamento p/ extrínsecos normais (75 %)	1.500.000.000.000 (1,5 s)
Peso por transferência (base + dispatch)	274.090.000
Transferências por bloco	≈ 5.472
Tempo de bloco	1 s
TPS (sustentável, blocos cheios)	≈ 5.472 TPS ✓

Comparação com spec 107:

- Spec 107: blocos de 6s, 2s de peso → ~912 TPS
- Spec 108: blocos de 1s, 2s de peso → **~5.472 TPS** (6× maior, só pela frequência)

Esta configuração **prioriza estabilidade**: mesma exigência de hardware do spec 107 (que já rodava sem problemas), mas com latência de confirmação 6× menor e TPS 6× maior.

5. Requisitos de hardware — Configuração conservadora ✓

Esta configuração foi escolhida para **rodar com segurança em hardware comum**.

5.1 O que o “peso” significa

No Substrate, `ref_time` é medido em **picossegundos de compute na máquina de referência** (`WEIGHT_REF_TIME_PER_SECOND = 1e12`). A máquina de referência do `polkadot-v0.9.40` é, aproximadamente, um **Intel Core i7-7700K (4 núcleos @ 4,2 GHz) com NVMe SSD**. Os pesos dos pallets (incluindo os ~174 M da transferência) foram calibrados nessa máquina — e são dominados por operações de **armazenamento** (RocksDbWeight: leitura ~25 M, escrita ~100 M).

5.2 A matemática da configuração conservadora

compute exigido por segundo = (peso do bloco / tempo do bloco)

Com blocos de 1s e orçamento de 2s de peso:

2 s de compute de REFERÊNCIA / 1 s de relógio = 2× a máquina de referência

Isso significa que cada validador precisa ser **~2x mais rápido** (single-core) que a máquina de referência. Com margem de segurança (usar só 50-70% do slot para executar+importar, reservando 30-50% para rede/propagação/consenso):

Utilização do slot	Speedup single-core exigido
50 %	~4,0x referência
70 %	~2,9x referência

5.3 Hardware comum atende

CPUs modernos comuns (2024/2025) têm desempenho single-thread de **~2,0-2,5x** o i7-7700K:

- **AMD Ryzen 7 7700X / 7800X3D:** ~2,3x referência
- **Intel i7-13700 / i9-13900:** ~2,5x referência
- **AMD Ryzen 9 7950X / Intel i9-14900K:** ~2,6x referência

Com ParityDB (mais rápido que RocksDB) e NVMe Gen3/4, a exigência cai para **~2-2,2x**, que esses CPUs entregam **confortavelmente**. Em blocos cheios de ~5.500 TPS, a utilização do slot fica em **50-70%**, com margem de segurança.

Conclusão: esta configuração **não exige hardware excepcional**. Um Ryzen 7 ou i7/i9 moderno com NVMe e 32 GB RAM roda tranquilamente. A cadeia **não trava** em hardware comum, ao contrário de uma configuração agressiva (4s de peso / 1s de bloco).

5.4 Requisitos de hardware recomendados

Para operar um **validador** com esta configuração conservadora:

Recurso	Mínimo	Recomendado
CPU	6-8 núcleos, single-thread ≥ 2x i7-7700K (Ryzen 5 7600 / i5-13600)	Ryzen 7 7700X / i7-13700 ou superior
RAM	16 GB	32 GB
Disco	NVMe SSD 512 GB	NVMe Gen4 1 TB, ParityDB
Rede	500 Mbps simétrico, <50 ms entre validadores	1 Gbps simétrico, baixa latência
DB backend	--database paritydb recomendado	--database paritydb + --state-pruning=archive-canonical

Esta configuração **roda em VPS comum** (ex.: Hetzner AX52, OVH Rise-2, AWS c7i.2xlarge).

5.5 E se quiser mais TPS no futuro?

Para ir além de ~5.500 TPS sem mudar o tempo de bloco:

1. **Re-benchmark dos pesos** em ParityDB/NVMe — pode reduzir peso/tx em ~30–50%, elevando o teto para ~7–8k TPS com o mesmo hardware.
2. **Mandato de hardware topo de linha** (Ryzen 9, Xeon/EPYC, NVMe Gen5) + orçamento de peso de 3–4s → ~8–11k TPS, mas exige centralização.
3. **Migrar para `polkadot-sdk`** (ver `docs/MIGRATION_GUIDE.md`) — abre caminho para otimizações futuras e execução mais eficiente.

6. Resumo — Configuração conservadora e segura

Objetivo	Situação	Observação
Taxa ~0,002 LUNES (transfer)	✓ Implementado	0,00200 LUNES; demais operações proporcionais
Demais taxas proporcionais	✓ Implementado	NFT 2,45×, contrato 13,76× — proporcionais ao peso
Tempo de bloco 1 s	✓ Implementado	Requer cadeia nova; slot não muda em cadeia viva
Capacidade ~5.500 TPS	✓ Implementado	~5.472 TPS sustentáveis com blocos cheios
Hardware comum	✓ Suportado	Ryzen 7 / i7 moderno + NVMe → sem risco de travar
Requisito de hardware	✓ Documentado	Ver seção 5.4; roda em VPS comum

Filosofia: esta configuração **não corre riscos**. Blocos 6× mais frequentes (1s vs 6s) mas com o mesmo orçamento de peso do spec 107 → TPS salta de ~912 para ~5.500 **apenas pela maior frequência**, sem exigir hardware excepcional. A cadeia roda com segurança em Ryzen 7, i7 moderno, NVMe e 32 GB RAM.

Recomendação: compilar, rodar `cargo test`, subir uma **testnet** com este runtime e medir a capacidade real sob carga. Para ir além de ~5.500 TPS no futuro, priorizar **re-benchmark dos pesos** em ParityDB/NVMe (ganho ~30–50%) antes de aumentar o orçamento de peso agressivamente.